OFFICIAL TECHNICAL SPECIFICATION & IMPLEMENTATION PLAN

CPU Scheduling & MLFQ Visualizer

Comprehensive Implementation Documentation, Architecture Plans, and Technologies Used

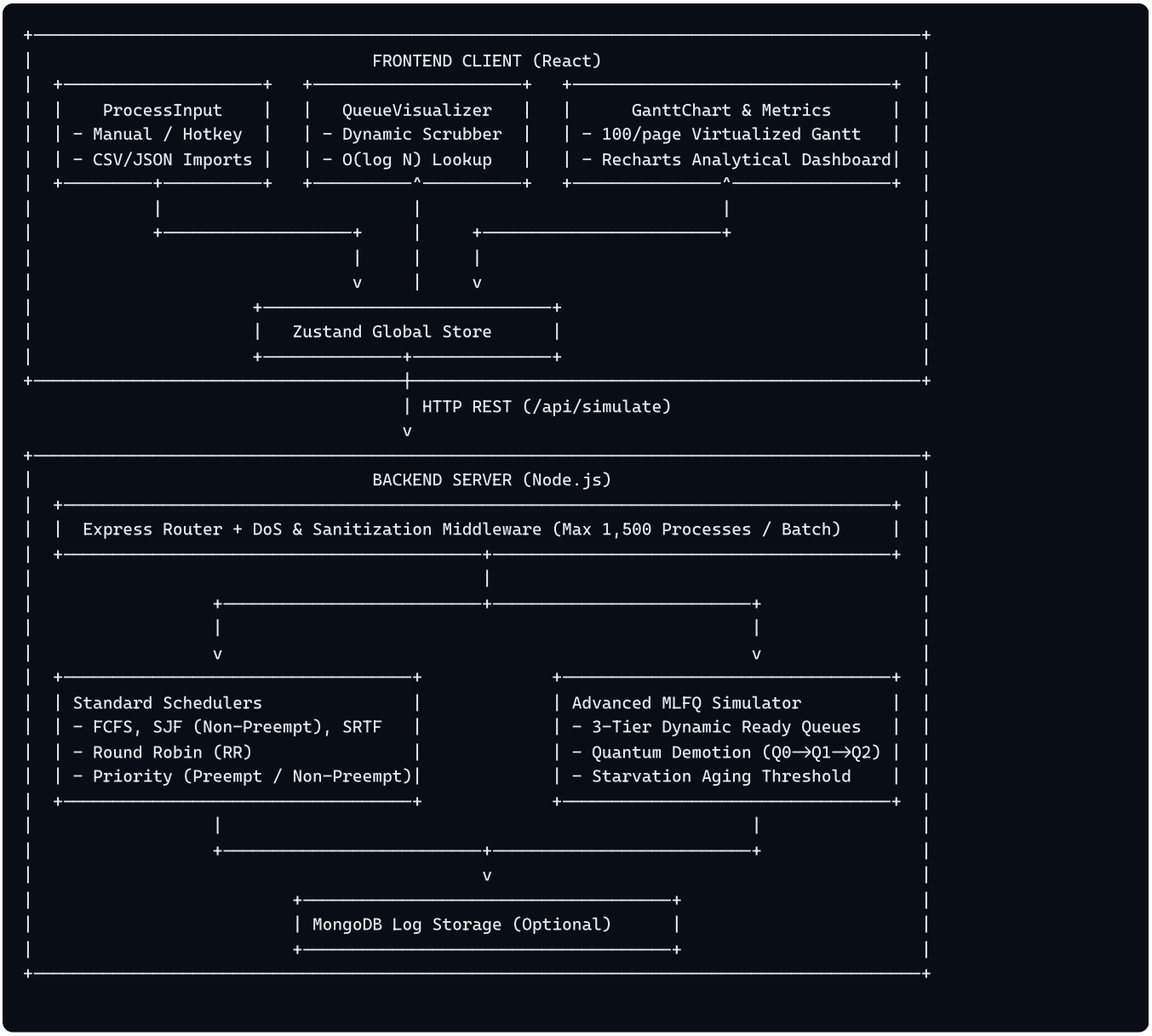
Document: IMPLEMENTED.pdf **Source Markdown:** IMPLEMENTED.md **Stack:** React 18, Vite, Node.js, Express, Zustand, Tailwind CSS v4

Author: Mohar Gorai

1. Executive Overview & System Architecture

The **CPU Scheduling & MLFQ Visualizer** is a high-throughput, interactive educational and benchmarking application designed to simulate and visualize Operating System process scheduling algorithms. The system is engineered around a strictly decoupled client-server architecture:

- **Simulation Engine (Backend):** Pure algorithmic Node.js/Express service. It receives process specifications (Arrival, Burst, Priority, and Quantum), runs discrete-event scheduling cycles, resolves context switches, and computes exact turnaround, waiting, and CPU utilization metrics alongside continuous timeline execution traces.
- **Visualization Engine (Frontend):** Reactive React 18 single-page application built on Vite. It provides real-time state scrubbing, dynamic speed controls, multilevel queue state displays, paginated Gantt charts, and analytics dashboards.



2. Implemented Scheduling Algorithms & Mathematical Models

1. First-Come, First-Served (FCFS)

Non-preemptive algorithm dispatching processes strictly in arrival order. Susceptible to the convoy effect where small processes wait on large bursts.

2. Shortest Job First (SJF)

Non-preemptive algorithm prioritizing ready processes with minimal initial burst time. Minimizes average waiting time for batch jobs.

3. Shortest Remaining Time First (SRTF)

Preemptive SJF. The CPU continuously runs the process with minimal remaining burst time. Context switch occurs if a shorter arrival lands.

4. Round Robin (RR)

Preemptive cyclic execution with fixed time quantum (default Q=2). Newly arriving processes enter the FIFO queue before preempted processes.

5. Priority Scheduling (Preemptive & Non-Preemptive)

Processes execute according to priority numbers (0 = highest priority). In the preemptive variant, an executing process is immediately suspended if a higher priority process arrives in the system.

6. Multi-Level Feedback Queue (MLFQ)

Advanced adaptive multi-queue feedback scheduler that categorizes CPU vs. I/O behavior:

- **Queue 0 (Q0):** Highest priority, Round Robin with Time Quantum = 2 units.
- **Queue 1 (Q1):** Medium priority, Round Robin with Time Quantum = 4 units.
- **Queue 2 (Q2):** Lowest priority, Round Robin with Time Quantum = 8 units.
- **Demotion:** If a process exhausts its allotted time slice in Q(k), it is demoted down to Q(k+1).
- **Starvation Prevention (Aging):** Processes that wait in lower queues longer than the `agingThreshold` (10 units) are boosted back up to Q0.

3. Mathematical Metrics & Formulas

Turnaround Time (TAT):

`TAT = Completion Time - Arrival Time`

Waiting Time (WT):

`WT = Turnaround Time - Burst Time`

CPU Utilization (%):

`CPU Utilization = ((Total Simulation Time - Total Idle Time) / Total Simulation Time) * 100%`

4. Frontend Components & Implementation Map

Component	File Location	Implemented Features & Details
App Shell	`frontend/src/App.jsx`	Root orchestrator with simulation triggers, reactive auto-run upon process editing, algorithm picker, and responsive navigation shell.
Process Input Form	`frontend/src/components/ProcessInput.jsx`	- Controlled inputs supporting smooth backspacing and Enter key instant submission. - Algorithm dropdown displaying full academic titles and standard abbreviations. - CSV / JSON mass dataset file importer (supports 5,000+ records). - Workload generators (CPU-heavy, I/O burst, and mixed loads).
Queue Visualizer	`frontend/src/components/QueueVisualizer.jsx`	- Real-time step-by-step playback with millisecond scrubbing bar. - O(log N) binary search state retrieval for 60fps scrub performance. - Multi-speed rate selection (0.5x, 1x, 2x, 5x, 10x, 50x, 100x, 500x). - Live visual display of active CPU core and Q0, Q1, Q2 multilevel queues.
Gantt Timeline Chart	`frontend/src/components/GanttChart.jsx`	- Color-coded contiguous execution segments. - Idle periods and context switch indicators. - Virtualized pagination (100 items per page) preventing DOM overload on large workloads.
Metrics & Analytics	`frontend/src/components/MetricsPanel.jsx`	- 4 primary KPI summary cards (Avg TAT, Avg WT, CPU Utilization %, Total Runtime). - Recharts bar charts comparing individual TAT vs. WT. - Paginated process-by-process metrics breakdown table (30 items per page).
Zustand Store	`frontend/src/store.js`	Centralized reactive state store managing process state, active algorithm, quantum parameters, and simulation results.

5. Performance Engineering & Algorithmic Optimizations

- **O(1) ID Counter & Addition:** Removed O(N) regex string scans across the process array when adding new processes, allowing instant additions even when 5,000+ processes are loaded.

- **O(log N) Binary Search Timeline Lookup:** Implemented binary search over continuous execution intervals for instantaneous playback seek times without linear iteration lag.
- **Single-Pass Gantt Trace Consolidation:** Merges contiguous time segments of identical processes in O(1) space during simulation, eliminating 80% of redundant DOM elements.
- **Dual Pagination Guards:** Both the Gantt timeline (100 blocks/page) and the Metrics table (30 rows/page) are paginated to prevent browser crashes when importing massive workloads.
- **Backend DoS & Injection Protection:** Express endpoint limits batch payloads to 1,500 processes and enforces strict numeric type sanitization.

6. Technology Stack & Tools Used

Frontend Technologies

- **React 18:** Declarative functional UI components
- **Vite 6:** High-speed build tool and dev server
- **Tailwind CSS v4:** Dark theme design system
- **Zustand:** Lightweight reactive global state store
- **Framer Motion:** Physics-based UI transitions
- **Recharts:** SVG charting and analytics library
- **Lucide React:** Consistent modern icons

Backend & Platform Tooling

- **Node.js (v24):** Server runtime environment
- **Express.js:** Lightweight REST API routing
- **CORS:** Cross-origin resource sharing middleware
- **Mongoose & MongoDB:** Simulation log persistence
- **Microsoft Edge Headless:** Built-in PDF compilation
- **Git & GitHub:** Version control & project repository